MostafaElAraby at the AraSeg Shared Task 2026: Combining Lexical and Punctuation Evidence across Aligned Text Views

Mostafa ElAraby
PhD Candidate, Mila, Quebec AI Institute
Université de Montréal
Montréal, Canada

Abstract

Arabic sentence segmentation without punctuation or paragraph breaks requires predicting boundaries from context. The AraSeg Open Track supplies a parallel punctuated view, but its marks do not consistently indicate sentence endings. We train two CAMeLBERT-MSA ensembles, one for the target No-Punctuation No-Paragraph view and one for the auxiliary No-Paragraph view. A discrete rule combines their boundary decisions with punctuation classes projected onto the target words. Model fitting and selection use only the 174-document official training split: five-fold cross-validation selects the training duration, and out-of-fold predictions calibrate the decoders and fusion rule. Development is reserved for evaluation, and the public test is unused. The system achieves 84.61 document-macro F1 on development and 84.88 on the official blind test, with blind precision of 81.87 and recall of 89.50. It predicts too many boundaries in 80.6% of development documents, although some of its lowest scores result from missed boundaries.

1 Introduction

Sentence segmentation determines where one sentence ends and the next begins. Arabic punctuation is inconsistent, and some texts omit sentence-final marks altogether. AraSeg evaluates segmentation across genres under different punctuation conditions (Elkholy et al., 2026a). Its No-Punctuation No-Paragraph (NoPnx-NP) condition removes punctuation and paragraph breaks, leaving the segmenter to infer boundaries from the words.

The Open Track permits a No-Paragraph (NP) view of the same documents that retains punctuation. A full stop in this view may end a sentence or occur within an abbreviation; an unmarked word may still end a sentence. We therefore combine punctuation with predictions from two CAMeLBERT-MSA ensembles that process the views independently.

Removing punctuation changes tokenization, so NP predictions must be mapped to the corresponding target words before they can be combined. We align the two views and represent each target word by three signals: its target-model decision, its projected NP decision, and its punctuation class. Their twelve possible combinations define a small rule space that can be searched exhaustively. The search uses document-disjoint out-of-fold (OOF) predictions from the training split.

The resulting rule can be inspected directly: it accepts NP boundaries and retains target-only boundaries when punctuation is absent. We describe how the models and rule are selected, report the official blind evaluation, and examine alignment failures and boundary-count errors on development data. Code and a Colab notebook for training and inference are available in the accompanying repository.[1]

2 Task and Related Work

The AraSeg shared task represents a document as tokens $w_1, \ldots, w_n$ and requires a binary decision $\hat{y}_i$, where $\hat{y}_i = 1$ denotes a sentence ending after w_i (Elkholy et al., 2026b). Precision, recall, and F1 are computed separately for each document and then averaged. We address the NoPnx-NP Open Track using NoPnx-NP as the target and NP as the auxiliary view. Hereafter, *development* denotes the 222-document dataset split, distinct from the 262-document public test.

Earlier Arabic segmentation methods combine punctuation with lexical cues. Hadrich Belguith et al. (2005) use contextual analysis of punctuation and particles for sentence and paragraph segmentation. Keskes et al. (2012) study clause segmentation using punctuation and lexical indicators. These tasks differ in granularity, but both motivate considering a mark in context rather than assigning it

[1] https://github.com/mostafaelaraby/Arabic-Sentence-Segmentation-Dual-View

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 906–911
October 28-29, 2026 ©2026 Association for Computational Linguistics

a fixed boundary interpretation.

BERT represents each token using bidirectional context (Devlin et al., 2019). CAMeLBERT adapts this architecture to Modern Standard Arabic (MSA), dialectal Arabic, Classical Arabic, and mixtures of these varieties (Inoue et al., 2021). We use its MSA checkpoint. Contextual classification lets the NP model interpret punctuation together with the surrounding words, while the target model must rely on the punctuation-free text.

3 System Description

Figure 1 shows the two classifiers, their decoders, and the projection into target word order.

3.1 Lexical models

We fine-tune CAMeLBERT-MSA with a two-class linear classification head for each view. The classifiers use the provided tokens without additional Arabic normalization. Each token's label is assigned to its final subtoken; the loss ignores the remaining subtokens. Documents are divided into windows of at most 512 subtokens, including special tokens, with a nominal overlap of 64 subtokens. Window boundaries are adjusted to avoid splitting words where possible. At inference, logits for words appearing in multiple windows are averaged before softmax.

Each view uses three models with seeds 42, 1337, and 2024. The decoder thresholds their mean probabilities, then scans from left to right, suppressing boundaries that would close a segment shorter than the selected minimum length. It optionally marks the final token as a boundary. If no boundary remains, it selects the token with the highest boundary probability. These last two operations can override the minimum-length constraint.

3.2 Train-only selection

For each view, we shuffle the 174 training documents with seed 42 and divide them into five folds of 34 or 35 documents. Each fold model trains on the other four folds for five epochs. After each epoch, its decoder is calibrated on the held-out fold, and the checkpoint with the highest document-macro F1 is retained. Ties retain the earlier checkpoint. The final training duration for that view is the rounded mean of the five selected epoch counts.

The retained checkpoints generate OOF probabilities for their held-out documents. Pooling these predictions gives one probability sequence per training document. We calibrate one decoder per view by searching thresholds from 0.05 to 0.95 in steps of 0.05, then refining within 0.04 of the best threshold in steps of 0.01. Both stages jointly consider minimum lengths of one to four tokens and whether to force a final boundary. Selection maximizes document-macro F1.

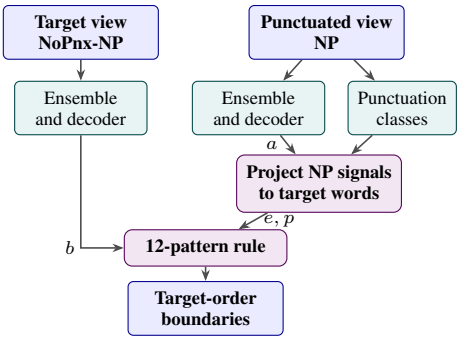


Figure 1: Two-view inference. The target decoder produces b; the NP decoder produces a in NP token order. Projection maps a and NP punctuation to target-order signals e and p, which the rule combines with b.

Three final models per view train on all 174 documents for the selected duration, without validation. They use the decoder calibrated on pooled OOF predictions. In total, ten fold models support selection and six final models produce the evaluation predictions.

3.3 Aligned fusion

Documents are paired by identifier. Alignment maps NP decisions and punctuation cues onto NoPnx-NP word positions, requiring every target word to match after punctuation normalization and the restricted deletions described below. A document enters fusion only when both projections succeed and have the target length. Unaligned training pairs are excluded from rule calibration, but remain in the separate classifiers' training splits. Development uses the target prediction as a fallback; blind inference requires complete alignment.

Projection handles punctuation as separate tokens, at word edges, or inside words. These transformations affect only matching; the classifiers retain their original inputs. Cues come from trailing marks and subsequent punctuation-only tokens. Terminal marks include Arabic and Latin question marks, full stops, exclamation marks, ellipses, and the Urdu full stop. Arabic and Latin commas and semicolons, the colon, and the Arabic thousands separator form the clause class. Terminal cues take precedence when several marks map to one word.

The alignment also permits deletion of source-only decimal integers and single alphabetic characters to accommodate enumerators removed from the target view. Other deletions, target insertions, and lexical substitutions are rejected. Predictions on punctuation-only NP tokens are combined with the preceding lexical token by taking their maximum. Thus a positive decision on a separate full stop becomes a boundary after the corresponding target word. Internal punctuation may be removed for matching but does not itself generate a punctuation cue.

For target word i, b_i is the decoded NoPnx-NP decision, e_i the aligned NP decision, and $p_i \in \{0, 1, 2\}$ the punctuation class: none, clause, or terminal. These signals define twelve patterns,

$$k_i = 3(2b_i + e_i) + p_i, \qquad k_i \in \{0, \ldots, 11\}. \quad (1)$$

A rule selects a subset S of patterns for which a boundary is predicted. We evaluate all $2^{12} = 4096$ subsets on OOF predictions for the 170 aligned training pairs. The objective is document-macro F1, with ties resolved in favor of fewer positive patterns. The selected set is $S = \{3, 4, 5, 6, 9, 10, 11\}$.

We score candidate rules from counts. For training document d and pattern k, let N_{dk} count its occurrences and T_{dk} count those with a reference boundary. If G_d is the total number of reference boundaries, a candidate rule has

$$F_{1,d}(S) = \frac{2\sum_{k\in S} T_{dk}}{G_d + \sum_{k\in S} N_{dk}}. \quad (2)$$

A zero denominator contributes zero. Averaging this expression over the aligned training documents gives the selection objective. Once the counts are computed, all candidate rules can be evaluated without further encoder passes.

3.4 Interpreting the rule

Table 1 gives the complete decision table. A positive auxiliary decision $e_i = 1$ produces a boundary for every punctuation class. When $e_i = 0$, a positive target decision is retained only if $p_i = 0$. If both models predict zero, punctuation alone does not create a boundary. Equivalently, the selected rule is $\hat{y}_i = e_i \vee (b_i \wedge [p_i = 0])$.

The asymmetric case $(b_i, e_i) = (1, 0)$ is estimated from training predictions. Although the search distinguishes clause and terminal marks, the selected rule assigns them identical outcomes. Only the presence of either punctuation class affects this rule.

b_i	e_i	None	Clause	Terminal
0	0	0	0	0
0	1	1	1	1
1	0	1	0	0
1	1	1	1	1

Table 1: The selected fusion decision for every evidence combination. Columns give p_i; entries are final boundary labels.

Two-view boundary inference

Input: target words x, NP words z, six fitted models, fixed decoders D_x, D_z, and rule S.

1. Compute $\bar{q}^{(x)}$ and $\bar{q}^{(z)}$ by window aggregation and seed averaging.
2. Obtain $b = D_x(\bar{q}^{(x)})$ and $a = D_z(\bar{q}^{(z)})$. Apply thresholding, minimum length, and the optional final boundary; if no boundary remains, select the highest-probability token.
3. Project a and punctuation from z onto x, yielding e and p only if both mappings succeed.
4. On a development alignment failure, return b; on a blind alignment failure, stop inference.
5. For every target word i, return $\hat{y}_i = \mathbf{1}[3(2b_i+e_i)+p_i \in S]$.

Figure 2: Inference with parameters selected exclusively from training data. The output has one decision per target word.

3.5 Prediction algorithm

Let $q_j^{(v,s)}$ denote the boundary probability at word j from seed s in view v, after window aggregation and softmax. The ensemble probability is

$$\bar{q}_j^{(v)} = \frac{1}{3} \sum_{s\in\{42,1337,2024\}} q_j^{(v,s)}. \quad (3)$$

Figure 2 specifies the remaining operations. Averaging precedes thresholding, and the NP decoder operates in NP token order before projection. Minimum-length and final-word settings apply within each lexical decoder. The fusion rule directly returns target labels without an additional decoding pass.

4 Experimental Setup

4.1 Data protocol

Table 2 summarizes the data splits. Only the two official training views contribute labels to model fitting, decoder calibration, and fusion-rule selection. Development is evaluated after all choices are fixed. The public test is unused, and the unlabeled blind views are used only for inference.

Training and development are checked for shared identifiers and identical token sequences within each view. Cross-validation partitions whole

Split	Docs	Words	Boundaries	Role
Train	174	98,457	10,247	Fit/select
Development	222	127,722	12,926	Held-out eval.
Public test	262	123,561	12,342	Unused
Blind test	212	96,798	–	Inference

Table 2: NoPnx-NP split sizes and their use in the experiments.

Evaluation	Precision	Recall	F1
Development	80.66	91.01	84.61
Blind test	81.87	89.50	84.88

Table 3: Document-macro precision, recall, and F1 (%). Blind scores are provided by the shared-task organizers.

Metric	25th pct.	Median	75th pct.
Precision	75.00	81.52	88.21
Recall	89.19	95.18	100.00
F1	80.49	85.71	91.30

Table 4: Quartiles of document-level development scores (%).

Split	Pairs	Aligned	Coverage (%)
Training	174	170	97.70
Development	222	218	98.20
Blind test	212	212	100.00

Table 5: Complete projection of both NP decisions and punctuation onto target words.

documents before windowing, so overlapping windows from one document cannot appear on both sides of a fold. The same fitted models, decoders, and rule are used for development and blind evaluation.

4.2 Optimization and evaluation

Optimization uses unweighted cross-entropy loss and AdamW (Loshchilov and Hutter, 2019), with learning rate 3×10^{-5}, weight decay 0.01, linear learning-rate decay without warmup, and gradient clipping at 1.0. Batches contain 16 windows; accumulation over two steps gives a nominal effective batch size of 32 windows. Inference uses PyTorch 2.11.0 (Paszke et al., 2019) and Transformers 4.48.3 (Wolf et al., 2020). Both classifiers start from CAMeLBERT-MSA, with no further task-specific corpus beyond the official training views.

For each document, precision and recall measure the boundary class, and F1 is their harmonic mean. Each metric is then averaged across documents, giving every document equal weight regardless of length. Undefined per-document ratios contribute zero. Macro F1 is therefore the mean of document F1 scores, not the harmonic mean of macro precision and recall. We report all scores as percentages.

The organizers supply the aggregate blind scores. Development statistics are computed from the saved per-document scores after selection is complete.

5 Results and Analysis

5.1 Overall performance

Table 3 reports 84.61 macro F1 on development and 84.88 on the blind test. Blind precision is 1.21 percentage points higher and recall is 1.51 points lower. Recall exceeds precision on both sets, but the gap is smaller on the blind test.

Median development F1 is 85.71, with an interquartile range of 80.49–91.30 (Table 4). Thirty documents (13.5%) score below 75 F1, including two below 50. Median recall is 95.18, compared with median precision of 81.52.

5.2 Alignment coverage

Table 5 reports complete projection of both auxiliary signals. All blind pairs align. The four unaligned development documents use the target-view fallback and remain in the evaluation; none is discarded.

The 218 aligned development documents achieve 84.91 macro F1, compared with 68.25 for the four fallback documents. Their count-weighted mean is 84.61. Because the groups contain different documents, this comparison does not measure the effect of fusion. The lowest fallback F1 is 47.06; the lowest overall F1, 44.44, occurs on an aligned document.

5.3 Development error profile

Let $\widehat{B}_d$ and B_d be the predicted and reference boundary counts for document d. If precision P_d and recall R_d are positive, then $\widehat{B}_d/B_d = R_d/P_d$. All development documents satisfy this condition. A surplus of predicted boundaries occurs in 179 documents (80.6%); 33 (14.9%) have a deficit, and ten (4.5%) have equal counts. Equal counts can conceal errors because false positives and false negatives may occur in equal numbers at different positions.

Of the 59 documents with perfect recall, 57 contain false-positive boundaries. Of the eleven with

perfect precision, nine contain missed boundaries. Only two documents have neither type of error.

The two extremes show different errors. The document with the lowest F1 has 100.00 precision and 28.57 recall (44.44 F1): every predicted boundary is correct, but most reference boundaries are missed. The document with the lowest precision has 37.68 precision and 96.30 recall (54.17 F1), or approximately 2.56 predicted boundaries per reference boundary. These examples are selected by their extreme scores.

The ratio identity applies within documents; the ratio of macro recall to macro precision does not recover corpus-wide boundary counts. The score summaries also do not locate errors. Boundary-level inspection is needed to distinguish clause boundaries, abbreviations, and other possible sources of disagreement.

6 Limitations and Ethics

The held-out folds select checkpoints and generate the OOF probabilities used to calibrate decoders and fusion. Their scores are therefore subject to selection bias, even though each document is excluded from gradient-based fitting for its fold model. Development and blind evaluation assess the selected system, but a single run does not quantify variation across training partitions or seed sets.

We do not report component ablations. The results therefore do not establish whether the auxiliary classifier, ensembling, or fusion improves over a simpler baseline. Comparisons would need to use common evaluation documents and restrict all selection to training data.

The full system requires a parallel punctuated view. Its performance does not characterize the target classifier used alone. Alignment also assumes that source-only integers and single letters can be omitted; this token-level criterion does not verify that every omission is semantically harmless. All blind pairs satisfy the criterion, but a future mismatch would stop inference under the strict policy used here.

Only aggregate blind scores are available, preventing document-level confidence intervals, error localization, and genre-specific blind analysis. The 0.27-point F1 difference from development is descriptive; we make no significance or ranking claim.

Segmentation errors may propagate to retrieval, translation, and other downstream applications. Use beyond AraSeg requires evaluation on the intended Arabic varieties and genres, along with checks that the parallel views correspond.

7 Conclusion

The two-view CAMeLBERT system achieves 84.88 macro F1 on the official blind test after fitting and selection on the training split alone. Its fusion rule uses punctuation to resolve disagreements between the two classifiers. Most development documents have too many predicted boundaries, although missed boundaries account for some of the lowest scores. Reducing false positives is a useful next step, provided it does not worsen those under-segmented cases.

References

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026a. Arabic sentence segmentation across genres and punctuation conditions. *Preprint*, arXiv:2606.08025. Accepted to Findings of EMNLP 2026.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026b. Arabic sentence segmentation shared task 2026. In *Proceedings of the Fourth Arabic Natural Language Processing Conference*, Budapest, Hungary. Association for Computational Linguistics.

Lamia Hadrich Belguith, Leila Baccour, and Ghassan Mourad. 2005. Segmentation de textes arabes basée sur l'analyse contextuelle des signes de ponctuations et de certaines particules. In *Actes de la 12ème conférence sur le Traitement Automatique des Langues Naturelles. Articles courts*, pages 451–456, Dourdan, France. ATALA.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Iskandar Keskes, Farah Benamara, and Lamia Hadrich Belguith. 2012. Clause-based discourse segmentation of Arabic texts. In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*, pages 2826–2832, Istanbul, Turkey. European Language Resources Association (ELRA).

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035. Curran Associates, Inc.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.